

Drumroll, please...

We have reached a major milestone!



Java - the programming language - is now pretty much covered

But there is more to it, a whole ecosystem of libraries, frameworks and tools for building modern applications better and faster.

We will now learn how to build an application using industry best practices and popular components from the Java Ecosystem.

Presenting...



JavaBank

A Bank? Why?

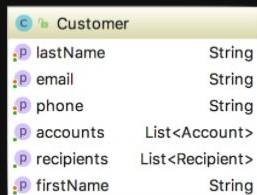


A banking system covers a lot of the functionality and responsibilities of a typical enterprise application, such as:

- User Interface
- Client/Server Architecture
- Relational Databases
- Database Transactions

Domain Entities

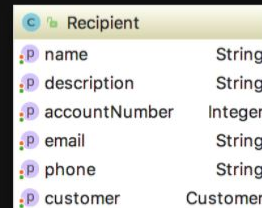
The JavaBank application deals with three main entities:



UML class diagram for the Customer entity. The class has a package icon, a name icon, and a list icon. It contains six attributes: lastName (String), email (String), phone (String), accounts (List<Account>), recipients (List<Recipient>), and firstName (String). Each attribute is preceded by a small icon indicating its type (e.g., a 'P' in a circle for primitive types, a 'C' in a circle for collections).

Customer	
lastName	String
email	String
phone	String
accounts	List<Account>
recipients	List<Recipient>
firstName	String

Customer: Holds one or more accounts



UML class diagram for the Recipient entity. The class has a package icon, a name icon, and a list icon. It contains six attributes: name (String), description (String), accountNumber (Integer), email (String), phone (String), and customer (Customer). Each attribute is preceded by a small icon indicating its type.

Recipient	
name	String
description	String
accountNumber	Integer
email	String
phone	String
customer	Customer

Recipient: receives money transfers from other customers



UML class diagram for the Account entity. The class has a package icon, a name icon, and a list icon. It contains three attributes: balance (double), customer (Customer), and accountType (AccountType). Each attribute is preceded by a small icon indicating its type.

Account	
balance	double
customer	Customer
accountType	AccountType

Account: Holds a balance

Functional Requirements

The main features of the JavaBank application are:

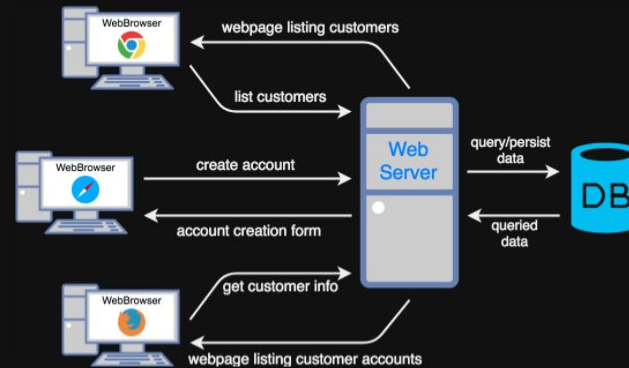
- **Customer Management:** Create, list, edit and delete customers
- **Account Management:** Create, list, edit and delete accounts
- **Recipients Management:** Create, list, edit and delete recipients
- **Money Transactions:** Withdraw, deposit and transfer money between accounts or to recipients

Technical Requirements

JavaBank should be split into
a client and server

The user interface should be a web
application - making it accessible
through a web browser

Data should be stored in a
persistent storage



Customer Management

Customer List

Id	First Name	Last Name	Email	Phone	Show	Edit	Delete
1	Rui	Ferrão	mail@gmail.com	777888	View	Edit	Delete
2	Sergio	Gouveia	mail@gmail.com	777888	View	Edit	Delete
3	Bruno	Ferreira	mail@gmail.com	777888	View	Edit	Delete
4	No Accounts	No name	mail@gmail.com	777888	View	Edit	Delete

[New Customer](#)

Account Management ...with money transactions!

The screenshot shows a web browser window with the title 'JavaBank' and the URL 'localhost:8080/javabank3/customer/1'. The page displays customer information for 'Customer #1' and a table of accounts.

Customer #1

First Name: Rui
Last Name: Ferrão
Email: mail@gmail.com
Phone: 777888

Accounts

[Add](#) [Transfer](#)

#	Balance	Type	Deposit	Withdraw	Close
1	100.0	CHECKING	Deposit	Withdraw	Close
2	50.5	SAVINGS	Deposit	Withdraw	Close

[Recipients](#) [Customer List](#)

Recipients Management ...with money transactions!

The screenshot shows a web browser window titled 'JavaBank' with the URL 'localhost:8090/javabank3/customer/1/recipients'. The page displays information for 'Customer #1' and a list of recipients.

Customer #1

First Name: Rui
Last Name: Ferrão
Email: mail@gmail.com
Phone: 777888

Recipients:

[Transfer](#)

Account	Name	Email	Phone	Description	Edit	Delete
3	Sergio Gouveia	sergio@gmail.com	777888	My colleague Sergio from A/C	Edit	Delete
5	Bruno Ferreira	bruno@gmail.com	777888	My colleague Bruno from A/C	Edit	Delete

[New Recipient](#) [Customer Page](#)

JavaBank Git Flow

Repositories

- <A/C_> repository with the solutions
- Local repository with our solutions and yours
- Remote repository with everything
- Local repository at home

```
► git remote -v
AC      brunoferreira@bootcamp.academiadecodigo.org:/var/git/javabank-lx.git (fetch)
AC      brunoferreira@bootcamp.academiadecodigo.org:/var/git/javabank-lx.git (push)
myRepo  brunoferreira@bootcamp.academiadecodigo.org:/home/brunoferreira/repos/myJavabankRepo (fetch)
myRepo  brunoferreira@bootcamp.academiadecodigo.org:/home/brunoferreira/repos/myJavabankRepo (push)
```

How your commit tree should look like...

```
* 64e9731 (HEAD -> master) pulled step 3
| * e892cb4 (mvc) mvc is awesome
| * 9d62b62 loving mvc
| * a35a298 adding mvc
|/
* 6a7f128 pulled step 2
| * 76ef948 (design-patterns) finished adding design_patterns
|/
* e5f929a pulled step 1
| * 5f28ffe (prompt-view) finished adding prompt-view
|/
* de85c52 cloned step 0
```

{ Exercise }

Get acquainted with the JavaBank code base
and create a UI using Prompt-View library